

Compilador

2025/2 - Compiladores

Matheus N. Toso

Linguagem

Linguagem

- Numérica
- Aritmética, lógica, relacionais
- For, ternário
- Booleanos numéricos
- Funções pré-definidas
- AST Top-Down
- Sem look ahead

Tokens

Tokens

- Vazio
- Nova linha
- Parênteses
- Números
- Operadores aritméticos (+, -, /, *, ^,)
- Operadores lógicos (&, |, #, !)
- Operadores relacionais (==, !=, >, <, >=, <=)
- Palavras reservadas
- Identificadores (^[a-zA-Z_]*\$)
- Operadores para ternário (:, ?)
- Vírgula

Gramática

Gramática

statements ::= newLine* <expression> (newLine+ <expression>)* newLine*

expression ::= var:key id assign <expressao>

| <logical-expression> question <logical-expression> colon <logical-expression>

| <logical-expression> (or | xor) <logical-expression>

| <logical-expression>

logical-expression ::= <relational-expression> and <relational-expression>

| <logical-expression> and <relational-expression>

| <logical-expression> and <logical-expression>

relational-expression ::= NEGACAO <relational-expression>

| <arithmetic-expression> (== | != | > | < | >= | <=) <arithmetic-expression>

| <arithmetic-expression> (== | != | > | < | >= | <=) <relational-expression>

| <relational-expression> (== | != | > | < | >= | <=) <relational-expression>

arithmetic-expression ::= <term> (+ | -) <term>

| <arithmetic-expression> (+ | -) <term>

| <arithmetic-expression> (+ | -) <arithmetic-expression>

Gramática

term ::= <operand> (* | /) <operand>
| <term> (* | /) <operand>
| <term> (* | /) <term>
| <power>

power ::= <call> (pow <term>)

call ::= <operand>
| <operand> lPar ((<expression> (comma <expression>)*?) rPar

operand ::= num | id
| (+ | -) operand
| lPar <term> rPar
| lPar <expression> rPar
| for

for ::= for:key id assign <expression> to:key <expression> (increment:key <expression>)? then:key
<expression>
| (newLine <statements> end:key)

Palavras Reservadas

Palavras Reservadas

- var
 - Declaração de variáveis
- for
 - Inicia loop for (em bloco ou linha)
- to
 - Fica entre o valor inicial de final do for
- increment
 - Permite modificar o incremento do for, suporta negativos
- then
 - Indica expressão a ser executada por iteração do for em linha
- end
 - Indica final do bloco para for em bloco

Funções

Funções

- `if(condition,true,false)`
- `sum(x,y,z,...,n)`
- `avg(x,y,z,...,n)`
- `max(x,y,z,...,n)`
- `min(x,y,z,...,n)`
- `abs(x)`
- `sqrt(x)`
- `rand(x)`

Compilação

Entrada

```
1  var a = rand(100)
2  var b = rand(100)
3    a>b ? a-b : b-a
4
5  var pi = 3.14
6  var r = 10
7  var C = 2*pi*r
8  var A = pi*r^2
9
10 var loopCount = 0
11
12 for i = C to A increment r then
13   var loopCount = loopCount + 1
14 end
15
16 loopCount
17
18 var k = -100
19 var result = 0
20
21 var loopCount = 0
22
23 for i = 0 to -100 increment -5 then
24   var loopCount = loopCount + 1
25   var result = sum(result,avg(result,k))
26 end
27
28 loopCount
29 result
30
```

Tokens

```
LEXER 1:-> [key::var, id::a, assign::=, id::rand, lPar::(, num::100, rPar::),
LEXER 2:-> , key::var, id::b, assign::=, id::rand, lPar::(, num::100, rPar::),
LEXER 3:-> , id::a, larger::>, id::b, question::?, id::a, minus::- , id::b, colon:::, id::b, minus::- , id::a,
LEXER 4:-> ,
LEXER 5:-> , key::var, id::pi, assign::=, num::3.14,
LEXER 6:-> , key::var, id::r, assign::=, num::10,
LEXER 7:-> , key::var, id::C, assign::=, num::2, mul::*, id::pi, mul::*, id::r,
LEXER 8:-> , key::var, id::A, assign::=, id::pi, mul::*, id::r, pow::^, num::2,
LEXER 9:-> ,
LEXER 10:-> , key::var, id::loopCount, assign::=, num::0,
LEXER 11:-> ,
LEXER 12:-> , key::for, id::i, assign::=, id::C, key::to, id::A, key::increment, id::r, key::then,
LEXER 13:-> , key::var, id::loopCount, assign::=, id::loopCount, plus::+, num::1,
LEXER 14:-> , key::end,
LEXER 15:-> ,
LEXER 16:-> , id::loopCount,
LEXER 17:-> ,
LEXER 18:-> , key::var, id::k, assign::=, minus::- , num::100,
LEXER 19:-> , key::var, id::result, assign::=, num::0,
LEXER 20:-> ,
LEXER 21:-> , key::var, id::loopCount, assign::=, num::0,
LEXER 22:-> ,
LEXER 23:-> , key::for, id::i, assign::=, num::0, key::to, minus::- , num::100, key::increment, minus::- , num::5, key::then,
LEXER 24:-> , key::var, id::loopCount, assign::=, id::loopCount, plus::+, num::1,
LEXER 25:-> , key::var, id::result, assign::=, id::sum, lPar::(, id::result, comma:::, id::avg, lPar::(, id::result, comma:::, id::k, rPar::), rPar::),
LEXER 26:-> , key::end,
LEXER 27:-> ,
LEXER 28:-> , id::loopCount,
LEXER 29:-> , id::result,
LEXER 30:-> , eof::]
```

AST

- Gerada pelo Parser
- O parser popula tabela de símbolos (id:nóValor)
- Cada nó principal é um statement/linha
- Cada statement pode conter múltiplos nós filhos

```

🌲 1: -> id::a
🌲 2: -> id::b
🌲 3: -> ((id::a, larger::>, id::b), question::?, (id::a, minus::- , id::b), colon:::, (id::b, minus::- , id::a))
🌲 4: -> id::pi
🌲 5: -> id::r
🌲 6: -> id::C
🌲 7: -> id::A
🌲 8: -> id::loopCount
🌲 9: -> (key::for, id::i, key::to, id::A, key::increment, Optional(id::r), key::then, [id::loopCount])
🌲 10: -> id::loopCount
🌲 11: -> id::k
🌲 12: -> id::result
🌲 13: -> id::loopCount
🌲 14: -> (key::for, id::i, key::to, minus::- , num::100, key::increment, Optional(minus::- , num::5), key::then, [id::loopCount, id::result])
🌲 15: -> id::loopCount
🌲 16: -> id::result
```


AST

- Compilada pelo Interpreter
- O Interpreter acessa a tabela de símbolos e resolve todos os nós da árvore

◉	INTERPRETER 1:->	48.660136030073076
◉	INTERPRETER 2:->	16.987559599318004
◉	INTERPRETER 3:->	31.672576430755072
◉	INTERPRETER 4:->	3.14
◉	INTERPRETER 5:->	10.0
◉	INTERPRETER 6:->	62.800000000000004
◉	INTERPRETER 7:->	314.0
◉	INTERPRETER 8:->	0.0
◉	INTERPRETER 9:->	0.0
◉	INTERPRETER 10:->	26.0
◉	INTERPRETER 11:->	-100.0
◉	INTERPRETER 12:->	0.0
◉	INTERPRETER 13:->	0.0
◉	INTERPRETER 14:->	0.0
◉	INTERPRETER 15:->	20.0
◉	INTERPRETER 16:->	-332425.6730079651

Tratamento de Erros

Tratamento de Erros

- Char ilegal
 - Genérico: Indica o char problemático
 - Número com mais de um ponto
- Runtime
 - Expressão inválida
 - Operador inválido
 - Variável não declarada no contexto atual
 - Parâmetros demais
 - Parâmetros faltando
 - Função não declarada
 - Radiciação inválida
 - Potenciação inválida
- Sintática inválida
 - Genérico: Token não pode ser utilizado após Token precedente
 - 'for' esperado
 - Identificador esperado
 - '=' esperado
 - 'to' esperado
 - 'then' esperado
 - 'end' esperado
 - ')' esperado
 - Número, id, '+', '-', ou '(' esperado
 - ')', var, número, id, '+', '-', '!' ou '(' esperado
 - ',' ou ')' esperado
 - Número, id, '+', '-', '!' ou '(' esperado
 - ':' esperado

```
LEXER      1:->    [key::var, id::a, num::123, eof::]
PARSER ERROR -> Error: Invalid Syntax: '=' expected.
File: shell | Line: 1 | Position: (6,9)

var a 123
      ^^^
```

```
LEXER      1:->    [num::1, plus::+, num::2, plus::+, eof::]
PARSER ERROR -> Error: Invalid Syntax: Number, identifier, '+', '-', '!' or '(' expected.
File: shell | Line: 1 | Position: (4,5)

1+2+
  ^
```

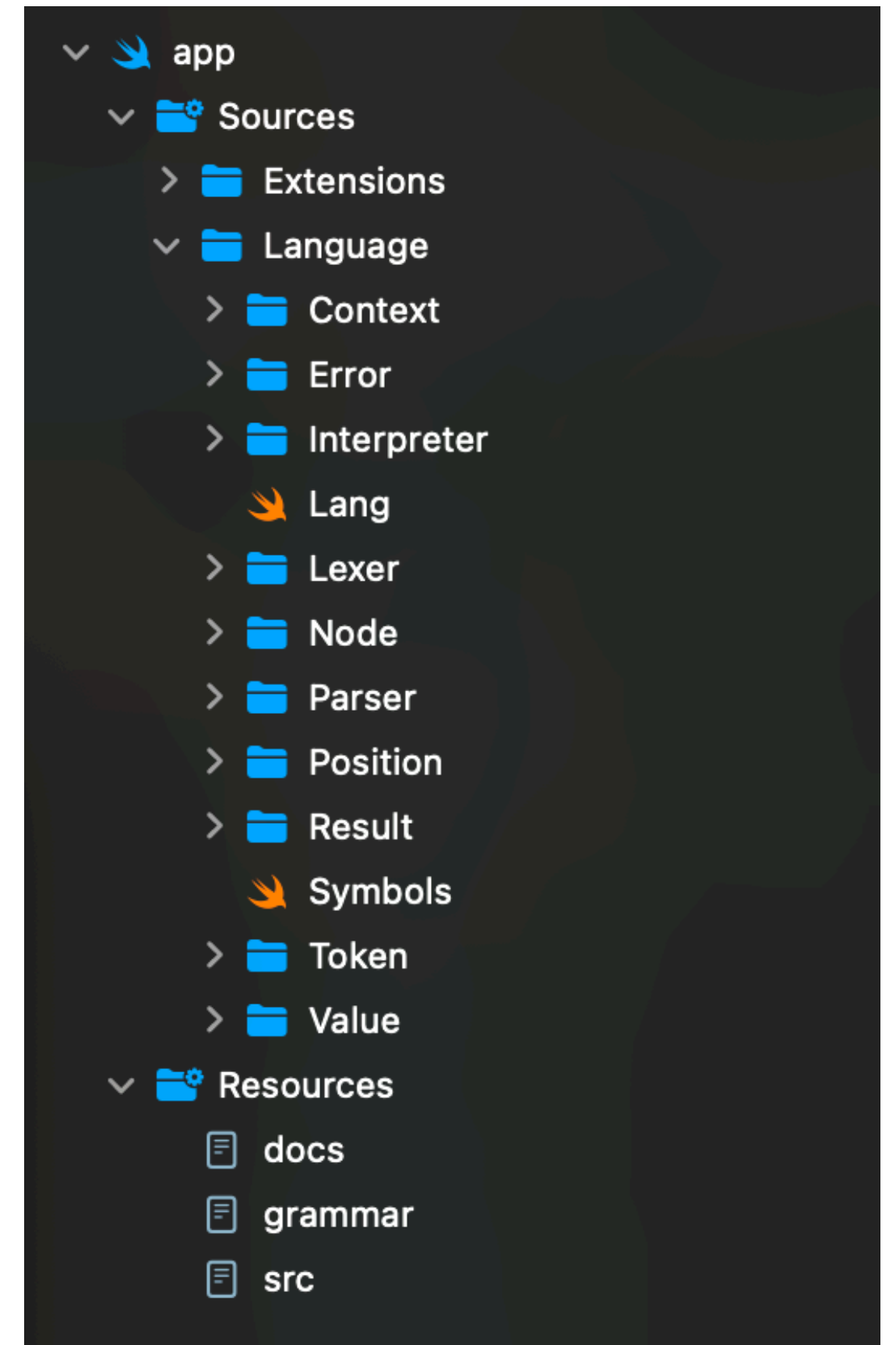
```
LEXER      1:->    [key::var, key::end, eof::]
PARSER ERROR -> Error: Invalid Syntax: Identifier expected.
File: shell | Line: 1 | Position: (4,7)

var end
      ^^^
```

Arquitetura

Arquitetura

- Swift
- 3 Camadas principais
 - Lexer
 - Tokens
 - Parser
 - Nós
 - Interpreter
- Contexto
 - Símbolos
 - Posição
 - Contexto pai
- Erros
- Resultados
- Compilação da string no arquivo “app”, e do arquivo src na pasta Resources



Limitações

Limitações

- Suporte somente a valores numéricos
- Não permite criar novas funções
- Sem look ahead